

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 5

Student Name: K.Rama

Roll No.: A24126552087

Section: CSM-B

Date: 22-09-2026

1. TITLE

DOM Demonstration — Changing Content, Style, and Attributes using JavaScript

2. AIM

To demonstrate three core ways JavaScript can manipulate the Document Object Model (DOM) — changing an element's content, changing its inline style, and changing its attributes — in response to separate button clicks, using HTML for structure, CSS for base styling, and JavaScript for DOM manipulation and event handling.

3. OBJECTIVE

The objectives of this experiment are:

- To select DOM elements using `getElementById()`.
- To change an element's text using the `textContent` property.
- To change an element's appearance at runtime using the `style` property.
- To add or modify an element's attributes using `setAttribute()`.
- To respond to button clicks using `addEventListener()`.

4. THEORY

The Document Object Model (DOM) represents an HTML page as a tree of objects that JavaScript can read and modify after the page has loaded. `document.getElementById()` retrieves a reference to a specific element using its id, after which its properties can be read or changed directly. The `textContent` property gets or sets the text inside an element, and assigning to it replaces the element's existing text entirely, which is the simplest way to change what a page displays. The `style` property gives direct access to an element's inline CSS; setting properties such as `style.color` or `style.fontSize` applies styling immediately, overriding any conflicting rule from an external stylesheet because inline styles carry the highest specificity. `setAttribute(name, value)` sets (or updates) any HTML attribute on an element, such as `title` (which produces a tooltip) or `class` (which changes which CSS rules from `style.css` apply to the element) — changing the `class` attribute this way lets a predefined CSS class, such as `.updated`, take effect without writing any inline styles. Each of the three demonstrations here is triggered independently by its own `addEventListener("click", ...)` call on a separate button, showing that DOM changes can be composed from small, focused event handlers.

5. ALGORITHM / PROCEDURE

1. Create `index.html` with a container `div` holding a heading (id "heading"), a paragraph (id "message") with placeholder text, and three buttons: "Change Content", "Change Style", and "Change Attribute".
2. Link an external `style.css` file and an external `script.js` file.
3. In `style.css`, style the container as a centred white card, style the message paragraph and buttons, and define an `.updated` class with a green colour, border, and background.
4. In `script.js`, select the heading, message, and all three buttons using `getElementById()`.
5. Attach a click listener to the "Change Content" button that sets `heading.textContent` and `message.textContent` to new text.

6. Attach a click listener to the "Change Style" button that sets `message.style.color`, `fontSize`, `fontWeight`, and `backgroundColor` directly.
7. Attach a click listener to the "Change Attribute" button that uses `setAttribute()` to add a title attribute and set the class attribute to "updated", then updates the message text to describe the change.
8. Open `index.html` in a browser and click each button in turn, observing that the page updates immediately without reloading.

6. CODE EXPLANATION

`index.html` defines a simple card-style container with a heading (id "heading"), a paragraph (id "message") holding placeholder text, and three separate buttons, each with its own id so `script.js` can attach an independent click handler to it. `style.css` centres the card on the page, gives the message paragraph readable padding and font size, styles the buttons, and defines an `.updated` class (green text, a green border, and a light green background) that is not applied to anything by default — it only takes effect once JavaScript adds it to the paragraph's class attribute. `script.js` selects all five elements up front with `getElementById()` and then wires up three independent click listeners. The "Change Content" handler simply reassigns `heading.textContent` and `message.textContent` to new strings, replacing the page's text outright. The "Change Style" handler reaches into `message.style` and sets four CSS properties directly (color, `fontSize`, `fontWeight`, `backgroundColor`), which take effect immediately as inline styles. The "Change Attribute" handler calls `setAttribute()` twice — once to add a title tooltip and once to set `class="updated"`, which brings the `.updated` CSS rule into effect — and then also updates the paragraph's text to explain what happened, showing that a single handler can combine several kinds of DOM change together.

7. PROGRAM

`index.html`

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>DOM Demonstration</title>

  <link rel="stylesheet" href="style.css">
</head>

<body>

  <div class="container">

    <h1 id="heading">DOM Demonstration</h1>

    <p id="message">
      This is the original paragraph.
    </p>

    <button id="contentButton">
```

```

        Change Content
    </button>

    <button id="styleButton">
        Change Style
    </button>

    <button id="attributeButton">
        Change Attribute
    </button>

</div>

<script src="script.js"></script>

</body>

</html>
style.css
body {
    font-family: Arial, sans-serif;
    background-color: #f2f2f2;
    text-align: center;
}

.container {
    background-color: white;
    width: 500px;
    margin: 100px auto;
    padding: 30px;
    border-radius: 10px;
    box-shadow: 0 0 10px gray;
}

#message {
    font-size: 18px;
    padding: 10px;
}

button {
    padding: 10px 15px;
    margin: 8px;
    cursor: pointer;
}

.updated {
    color: green;
    border: 2px solid green;

```

```
    background-color: #e8ffe8;
}
script.js
const heading = document.getElementById("heading");

const message = document.getElementById("message");

const contentButton =
    document.getElementById("contentButton");

const styleButton =
    document.getElementById("styleButton");

const attributeButton =
    document.getElementById("attributeButton");

contentButton.addEventListener("click", function() {

    heading.textContent = "Welcome to DOM";

    message.textContent =
        "JavaScript has changed the content dynamically.";

});

styleButton.addEventListener("click", function() {

    message.style.color = "blue";

    message.style.fontSize = "24px";

    message.style.fontWeight = "bold";

    message.style.backgroundColor = "lightyellow";

});

attributeButton.addEventListener("click", function() {

    message.setAttribute(
        "title",
        "This paragraph was modified!"
    );

    message.setAttribute(
```

```

        "class",
        "updated"
    );

    message.textContent =
        "The attributes of this paragraph have been changed.";

});

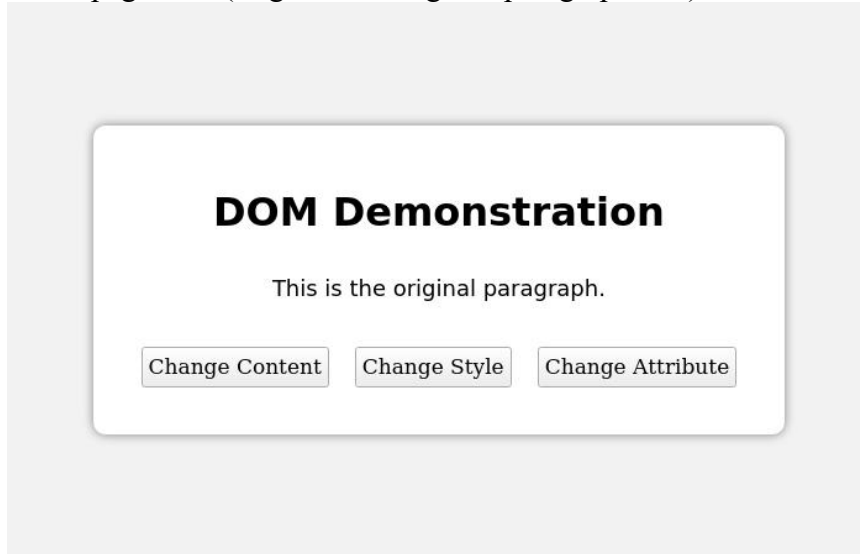
```

8. TEST CASES AND EXECUTION DATA

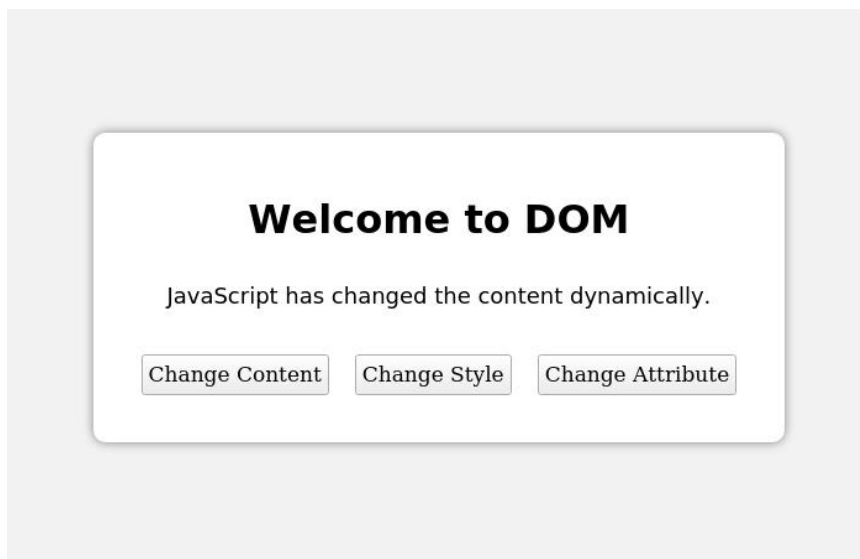
Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Load the page	Heading shows "DOM Demonstration" and the paragraph shows the original placeholder text	Page rendered correctly with original content	Pass
TC-02	Click "Change Content"	Heading changes to "Welcome to DOM" and the paragraph text changes to describe the content change	Heading and paragraph text updated correctly	Pass
TC-03	Click "Change Style"	Paragraph text turns blue, bold, larger (24px), with a light yellow background	Inline styles applied correctly to the paragraph	Pass
TC-04	Click "Change Attribute"	Paragraph gets a title tooltip, its class changes to "updated" (green border/background), and its text updates	Attributes and class applied; paragraph text updated	Pass
TC-05	Hover over the paragraph after clicking "Change Attribute"	Browser tooltip shows "This paragraph was modified!"	Tooltip displayed correctly from the title attribute	Pass

9. OUTPUT

Initial page load (original heading and paragraph text):



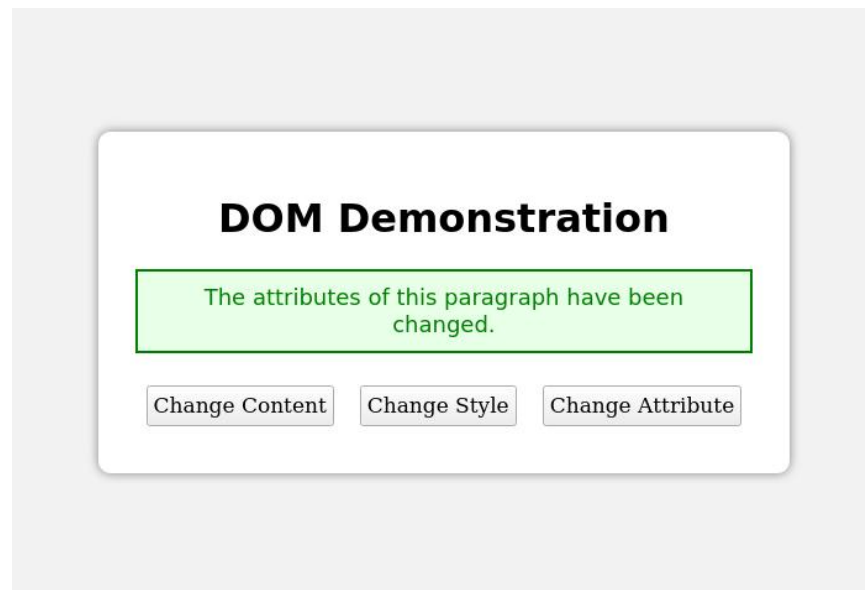
After clicking "Change Content" (heading and paragraph text replaced):



After clicking "Change Style" (paragraph restyled to blue, bold, larger text on a light yellow background):



After clicking "Change Attribute" (title tooltip added, class changed to "updated", text updated):



10. RESULT

Thus, a DOM demonstration page was successfully built using HTML, CSS, and JavaScript, showing three independent ways to manipulate the DOM at runtime: changing text content, changing inline styles, and changing attributes. Each button correctly triggered its own change without reloading the page, and the output was verified in a browser.